COWRIE SSH HONEYPOT

Home Lab Setup, Validation, and Findings

Environment: Kali Linux in Oracle VirtualBox
Client: Windows PowerShell OpenSSH
Honeypot: Cowrie 3.1.3
Validation date: 23 October 2026

Prepared by: Glennford S. Atay

Scope and Safety Statement

This project was conducted as a controlled cybersecurity home lab using Cowrie, an SSH honeypot, inside a Kali Linux virtual machine. The purpose was to deploy a simulated SSH service, validate connectivity from the Windows host, interact with the emulated shell, and inspect the events recorded by the honeypot. The work was limited to systems managed by the author.

The Windows host connected to Cowrie through VirtualBox NAT port forwarding. The client used 127.0.0.1:2222, a loopback endpoint on the Windows machine, while Cowrie listened on TCP port 2222 inside the Kali guest. This design supports local validation without intentionally making the service available to other devices or the public internet.

Cowrie is designed to emulate an SSH-accessible environment. The prompt, apparent hostname, operating-system details, and command responses shown in the session are honeypot output. They must not be treated as proof that the client accessed a real Debian server or a production network.

The successful login and command events in this report were produced by the author's own test session. They are evidence that the lab workflow and logging operated as expected, not evidence of an external attacker or real-world compromise. The original evidence included a test credential in a log line; that credential has been redacted from the portfolio copies of the screenshots.

For safe operation, the VirtualBox host-side forwarding rule should remain bound to 127.0.0.1 for this exercise. Public exposure or access from other machines is outside the scope of this report and should only be considered under a separately authorized plan with appropriate isolation and monitoring.

Executive Summary

This report presents the deployment and functional validation of Cowrie 3.1.3 in a Kali Linux virtual machine hosted by Oracle VirtualBox. Windows PowerShell and its OpenSSH client were used to test the service through a VirtualBox NAT port-forwarding rule. The lab was designed to demonstrate a complete basic workflow: service startup, port verification, host-to-guest connectivity, simulated SSH interaction, and event-log review.

On Kali Linux, the command `sudo ss -lntp | grep ':2222'` showed a TCP listening socket on `0.0.0.0:2222`, managed by the `twistd` process. On Windows, `Test-NetConnection 127.0.0.1 -Port 2222 -InformationLevel Detailed` returned `TcpTestSucceeded: True`. An SSH session was then established using `ssh -p 2222 root@127.0.0.1`, and the simulated prompt appeared as `root@svr04:~#`.

The commands `whoami`, `pwd`, `ls`, and `uname -a` were entered during the session. Cowrie's main log recorded connection and session activity, a successful simulated login event, and command records. The JSON log provided structured event data for review. Together, these results show that the local honeypot was reachable and that activity was captured. They do not establish access to a real server, nor do they represent attacker behavior outside the controlled lab.

1. Objectives and Scope

1.1 Project Objectives

The primary objective was to build a small, reproducible honeypot environment and demonstrate how a security analyst can validate a listening service and inspect the resulting activity. The specific objectives were:

- Install and initialize Cowrie in a Python virtual environment on Kali Linux.
- Start the service and verify that its SSH listener is active on TCP port 2222.
- Use VirtualBox NAT port forwarding to provide a local Windows-to-Kali connection path.
- Confirm connectivity from Windows with a TCP test and an SSH client session.
- Run harmless commands in the simulated shell to generate observable honeypot events.
- Inspect both the human-readable log and the structured JSON event log.

- Prepare screenshots and written findings that clearly distinguish simulated activity from real-world intrusion.

1.2 Project Scope

The work covered the Cowrie service lifecycle, the local virtual networking path, SSH client interaction, basic simulated-shell commands, and inspection of log evidence. The environment consisted of one Kali Linux virtual machine and the Windows host running VirtualBox. The test was performed manually and was intended to verify the setup rather than measure the behavior of real attackers.

1.3 Out of Scope

- Scanning, testing, or attempting to access production systems or third-party networks.
- Publishing the honeypot service to the internet or inviting unapproved external connections.
- Collecting real user credentials, private data, or sensitive information.
- Claiming that the observed login or commands came from an unknown attacker.
- Using the results to estimate real-world phishing, compromise, or attack success rates.

2. Lab Architecture and Configuration

2.1 Environment Overview

Kali Linux hosted the Cowrie service, while Windows PowerShell acted as the client. Cowrie was configured to accept SSH connections on guest TCP port 2222. VirtualBox NAT port forwarding provided a host-side entry point at 127.0.0.1:2222 and forwarded the connection to the guest service. This allowed the test to be initiated from Windows without relying on direct inbound connectivity to the guest's NAT address.

Component	Configuration / role	Purpose in the lab
Host operating system	Windows; PowerShell and OpenSSH client	Initiated the connectivity test and SSH session.
Virtualization	Oracle VirtualBox; Adapter 1 set to NAT	Provided an isolated virtual machine and forwarding path.
Guest operating system	Kali Linux	Hosted Cowrie and its logs.
Honeypot software	Cowrie 3.1.3	Emulated an SSH service and recorded interaction.
Guest listener	TCP 2222; process shown as twistd	Accepted forwarded SSH traffic inside Kali.
Host endpoint	127.0.0.1:2222	Local Windows endpoint used by the SSH client.
Observed source in Cowrie logs	10.0.2.2	Consistent with the VirtualBox NAT gateway address.

2.2 Network Flow

Windows PowerShell → 127.0.0.1:2222 → VirtualBox NAT forwarding → Kali TCP 2222 → Cowrie

The loopback address 127.0.0.1 refers to the Windows host itself. VirtualBox forwards traffic received on the configured host port to Cowrie inside the guest. Inside Kali, the listener appeared as 0.0.0.0:2222, which means the process was bound to all IPv4 interfaces available in the guest. The host-side forwarding rule remains an important boundary: binding it to 127.0.0.1 keeps the intended access path local to the Windows machine.

2.3 Cowrie Initialization and Service Startup

Cowrie was cloned into the user's home directory at `~/cowrie`. A Python virtual environment was created and used to install the project dependencies. The `cowrie init` command generated the configuration file and required runtime directories. The service was then started with `cowrie start`. Startup output and the main log indicated

that Cowrie 3.1.3 loaded its configuration and that the SSH factory was ready to accept connections on port 2222.

The use of a virtual environment keeps the application's Python dependencies separate from the system Python installation. The generated configuration and runtime directories provide locations for service settings, logs, session data, and other runtime artifacts. For reproducibility, future iterations should record any non-default configuration changes alongside the report.

2.4 Listener Verification

The command `sudo ss -lntp | grep ':2222'` was run in a Kali terminal. The output contained a `LISTEN` socket on `0.0.0.0:2222` and identified `twistd` as the owning process. This establishes that a process was bound to the expected TCP port at the time of the screenshot. It does not, by itself, prove that the Windows host can reach the service; that was verified separately using PowerShell.

3. Test Procedure

The following procedure was used to validate the lab from the service layer through the client interaction and event logging layers.

1. **Verify the listener:** In Kali, run `sudo ss -lntp | grep ':2222'`. Confirm that the output shows `LISTEN` on port 2222 and that the process is associated with Cowrie's runtime (`twistd`).
2. **Test Windows connectivity:** In PowerShell, run `Test-NetConnection 127.0.0.1 -Port 2222 -InformationLevel Detailed`. The key field is `TcpTestSucceeded`; the recorded output showed `True`, confirming that a TCP connection could be established to the local forwarded port.
3. **Open an SSH session:** Run `ssh -p 2222 root@127.0.0.1`. The first connection prompted for host-key confirmation, after which the client prompted for a password and displayed the simulated server banner and shell prompt.
4. **Execute benign commands:** At the simulated prompt, run `whoami`, `pwd`, `ls`, and `uname -a` individually. These commands were selected because they are basic inspection commands and do not intentionally modify the honeypot.
5. **Review the main log:** In a separate Kali terminal, inspect `~/cowrie/var/log/cowrie/cowrie.log`. The log includes readable messages for connection establishment, login attempts, and recognized commands.
6. **Review the JSON log:** Inspect `~/cowrie/var/log/cowrie/cowrie.json`. This file stores structured events that can be searched or processed by tools for later analysis.
7. **Filter relevant evidence:** Use grep to isolate connection, login, and command records. Filtering makes the key events easier to review and capture, but should be interpreted alongside the full logs when doing a more detailed investigation.

4. Results and Findings

All core validation checks shown in the supplied evidence passed. The results below summarize what each check demonstrates and the limits of the conclusion that can be drawn from it.

Validation item	Observed result	Assessment and meaning
Cowrie service	LISTEN on 0.0.0.0:2222; process shown as twistd	Passed. A process was listening on the expected guest port.
Windows TCP test	TcpTestSucceeded: True to 127.0.0.1:2222	Passed. The host could reach the forwarded local TCP endpoint.
SSH interaction	Prompt displayed as root@svr04:~#	Passed. The client established an SSH session with Cowrie's emulated environment.
Command execution	whoami, pwd, ls, and uname -a were entered	Passed. The session generated command activity for log validation.
Main log	Connection, login attempt, CMD, and Command found entries	Passed. Human-readable event records were produced.

JSON log	Structured SSH connection/session events present	Passed. Machine-readable event records were available for review.

4.1 Simulated Command Results

Command	Observed output	Interpretation
`whoami`	`root`	The emulated shell reported the root username. This is simulated identity information, not evidence of root access on the Kali guest or a production server.
`pwd`	`/root`	The emulated working directory was shown as /root.
`ls`	No output displayed	The emulated directory listing appeared empty in this session. No conclusion should be drawn about files on the real Kali system.
`uname -a`	Linux svr04 ... Debian 6.1.90-1 ... x86_64 GNU/Linux	Cowrie returned Linux-like system information for the simulated host.

These command results demonstrate that the session was interactive enough to return plausible shell output. They are not an independent inventory of the host operating system because Cowrie's purpose is to emulate the environment presented to the connecting client.

4.2 Main Log Findings

The main log showed three observed SSH connection events during the testing window. One session, identified in the evidence as `96da95df671c`, included the successful simulated login event and subsequent command records. The log contained `CMD` entries for `whoami`, `pwd`, `ls`, and `uname -a`, along with corresponding `Command found` messages for recognized commands.

The `New connection` entries record that a client reached the SSH service and provide session and endpoint details. The login-attempt entry records the authentication result within the honeypot. Command entries show what was typed into the simulated shell, while `Command found` indicates that Cowrie recognized the command. These records are useful for correlating a session's timeline and verifying that the logging path worked.

4.3 JSON Log Findings

The JSON log stores events as structured records rather than as only free-form text. The visible evidence includes fields such as a session identifier, protocol, source address and port, destination address and port, event identifier, and timestamp. Structured logs are easier to parse consistently and can support future filtering, event counting, or ingestion into a log-analysis platform.

The screenshots show that connection and session events were written to the JSON file. A complete event-by-event analysis would require reviewing the full JSON records and matching them by session identifier and timestamp; this report limits its conclusions to the fields and events visible in the supplied evidence.

4.4 Network Source Address

Cowrie logged the connection source as `10.0.2.2`. This address is consistent with the VirtualBox NAT gateway seen by a guest receiving forwarded traffic. It should not be described as the Windows host's Wi-Fi address, which was shown separately as `192.168.1.67` in an earlier connectivity test. Network address translation can make the source observed by the guest differ from the host's physical network address.

5. Skills Demonstrated

This lab demonstrates practical introductory skills relevant to security operations, system administration, and security testing. The skills are limited to the tasks actually performed and evidenced in the screenshots.

- Linux command-line administration: navigating a project directory, running service commands, and inspecting runtime files.
- Python environment management: creating and using a virtual environment for a Python-based application.
- Service verification: using `ss` to inspect listening TCP sockets and identify the associated process.
- Virtual networking: understanding the relationship between a guest service, VirtualBox NAT, and a host loopback endpoint.
- Windows command-line testing: using `Test-NetConnection` and the OpenSSH client from PowerShell.
- Honeypot validation: interacting with an emulated SSH shell without claiming that it is a real production server.
- Log analysis fundamentals: reviewing readable service logs, filtering relevant events with grep, and inspecting JSON records.
- Technical documentation: organizing test evidence, explaining observed results, and recording limitations accurately.

6. Limitations and Security Considerations

The findings should be interpreted within the boundaries of a single-machine lab. A successful local TCP test and SSH session demonstrate that the forwarding path and honeypot were functioning at the time of testing. They do not demonstrate that the service is secure against all forms of abuse or that the network configuration would behave identically in a different environment.

- Emulation boundary: Cowrie presents a simulated environment; command responses and system details do not prove access to a real Debian host.
- Test-generated activity: the connection, login, and commands were produced by the author's own validation session, not by an external attacker.
- NAT visibility: the guest's observed source address can reflect VirtualBox NAT behavior rather than the Windows host's physical interface address.
- Limited test coverage: the lab did not assess real-world attacker behavior, detection effectiveness under sustained activity, or the security posture of an organization.
- Credential hygiene: test credentials should be unique and should never be reused from personal or production accounts. Credentials should be redacted from public evidence.
- Exposure control: the host-side forwarding rule should remain bound to 127.0.0.1 for this local demonstration. Publicly exposing a honeypot introduces additional risk and is not necessary to reproduce these results.
- Evidence limits: screenshots capture selected moments. They are useful supporting artifacts but are not a substitute for preserving full logs and configuration when reproducibility is required.

7. Recommendations

The current setup is suitable as a basic portfolio demonstration. The following improvements would make future iterations easier to reproduce, audit, and explain.

8. **Preserve configuration and evidence:** Keep a dated copy of the Cowrie configuration, relevant logs, and screenshots. Record which settings were changed from defaults so another person can reproduce the environment.
9. **Use isolated test credentials:** Use credentials created specifically for the lab and avoid reusing real passwords. Before publishing screenshots, inspect all terminal output for secrets, usernames, host details, or tokens that should not be public.

10. **Maintain local-only access:** Keep the VirtualBox host-side port-forwarding bind address at 127.0.0.1 for this exercise. If another access pattern is required in the future, document the authorization, isolation, and exposure controls before changing the rule.
11. **Correlate events by session and time:** When extending the analysis, match the same session identifier across cowrie.log and cowrie.json, then compare timestamps to create a concise event timeline.
12. **Separate validation from threat claims:** Describe the work as controlled honeypot validation. Do not label the self-generated connection as attacker activity or use it to claim real-world detection performance.
13. **Expand testing carefully:** Future work could include testing service restart behavior, verifying that logs persist after restart, or creating a small script to summarize test-session events. Keep these tests inside the isolated lab.

8. Conclusion

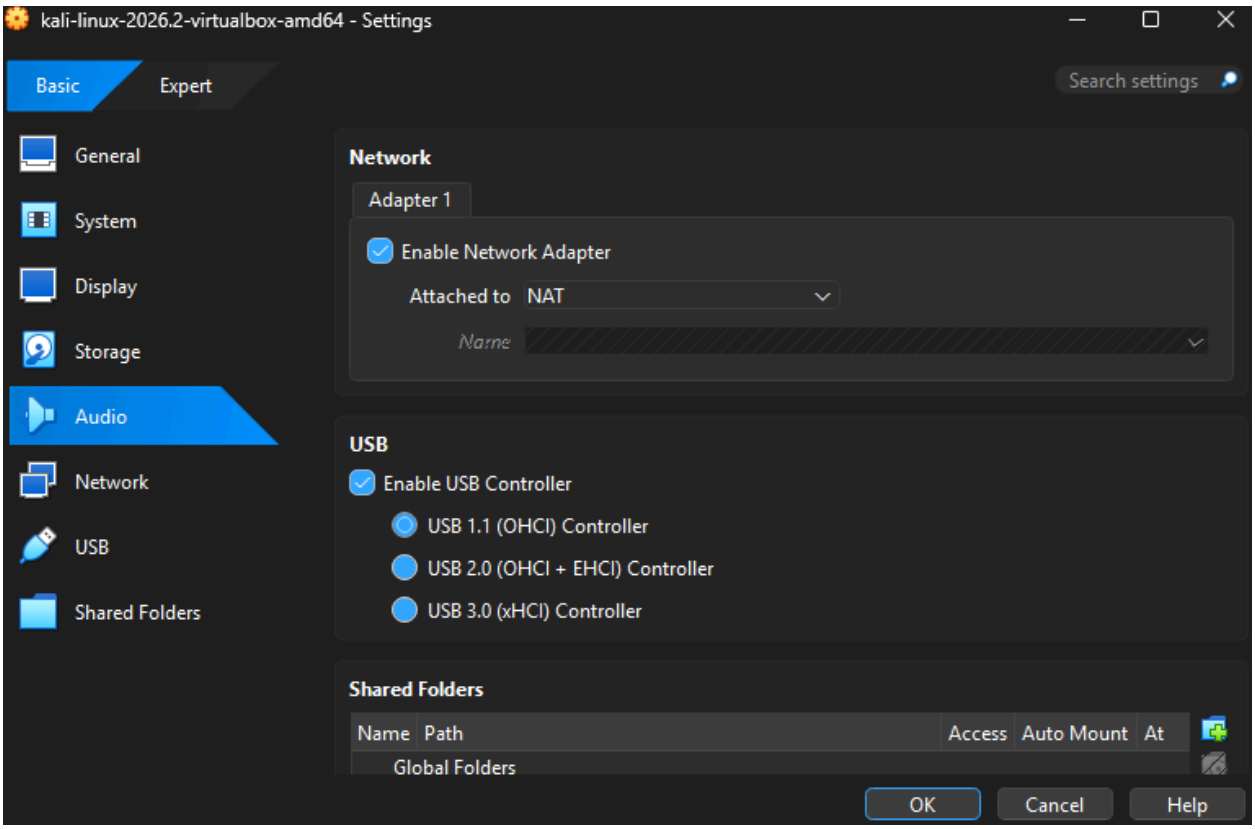
The Cowrie SSH honeypot home lab passed the core validation checks shown in the collected evidence. Cowrie was initialized and started in Kali Linux, a listener was confirmed on TCP port 2222, Windows reached the local forwarded endpoint, and an SSH client established a session with the simulated environment. The commands `whoami`, `pwd`, `ls`, and `uname -a` produced responses and generated records in Cowrie's main and JSON logs.

The work demonstrates a functioning local honeypot workflow, basic virtual networking, SSH client usage, and introductory event-log analysis. Its main value is the end-to-end validation and clear evidence trail. Because Cowrie emulates the server environment and the activity was generated by the author, the results must be presented as a controlled technical simulation rather than real server access or an observed attack.

Appendix A. Screenshot Evidence

The figures below reproduce the screenshots supplied during the lab. Captions state what each image supports. The credential visible in original authentication-log evidence has been redacted in the copies used here.

Figure A1. VirtualBox network mode



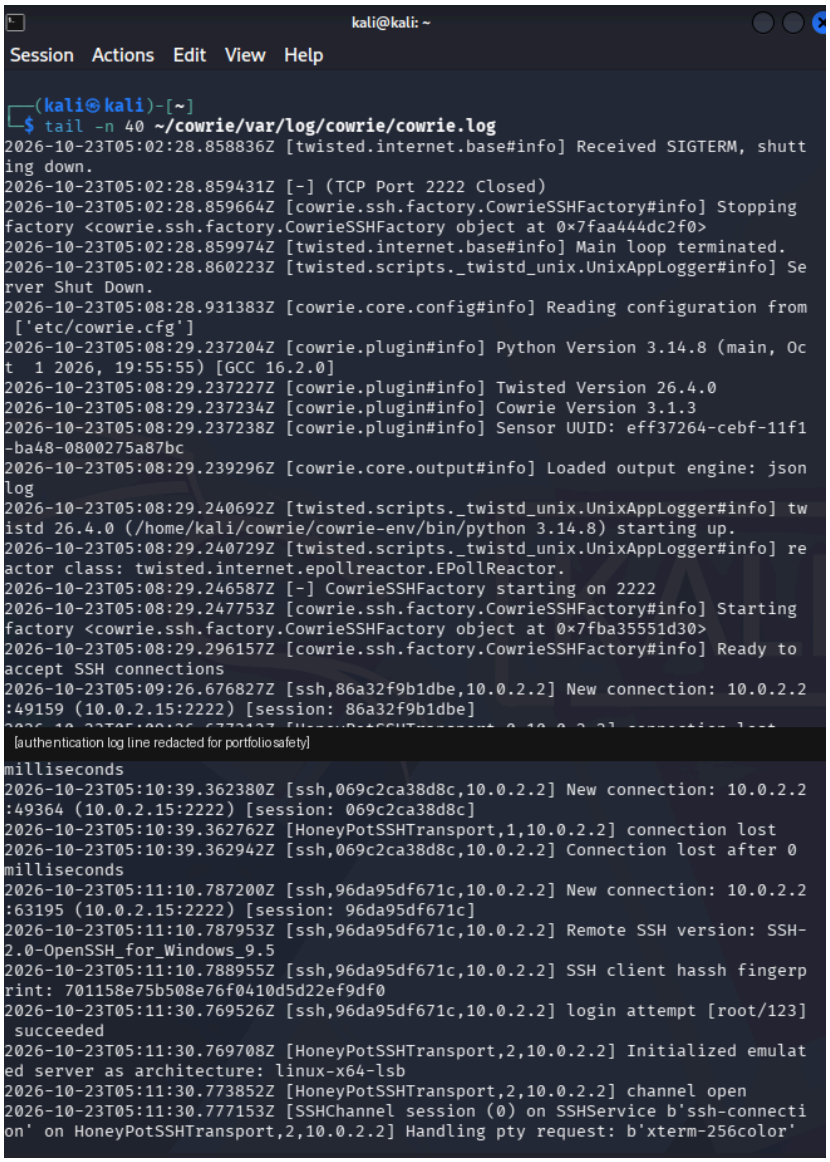
VirtualBox settings show the virtual machine's network adapter attached to NAT. This documents the chosen network mode; the separate successful TCP and SSH tests provide functional evidence that the forwarding path worked.

Figure A2. Cowrie JSON event log



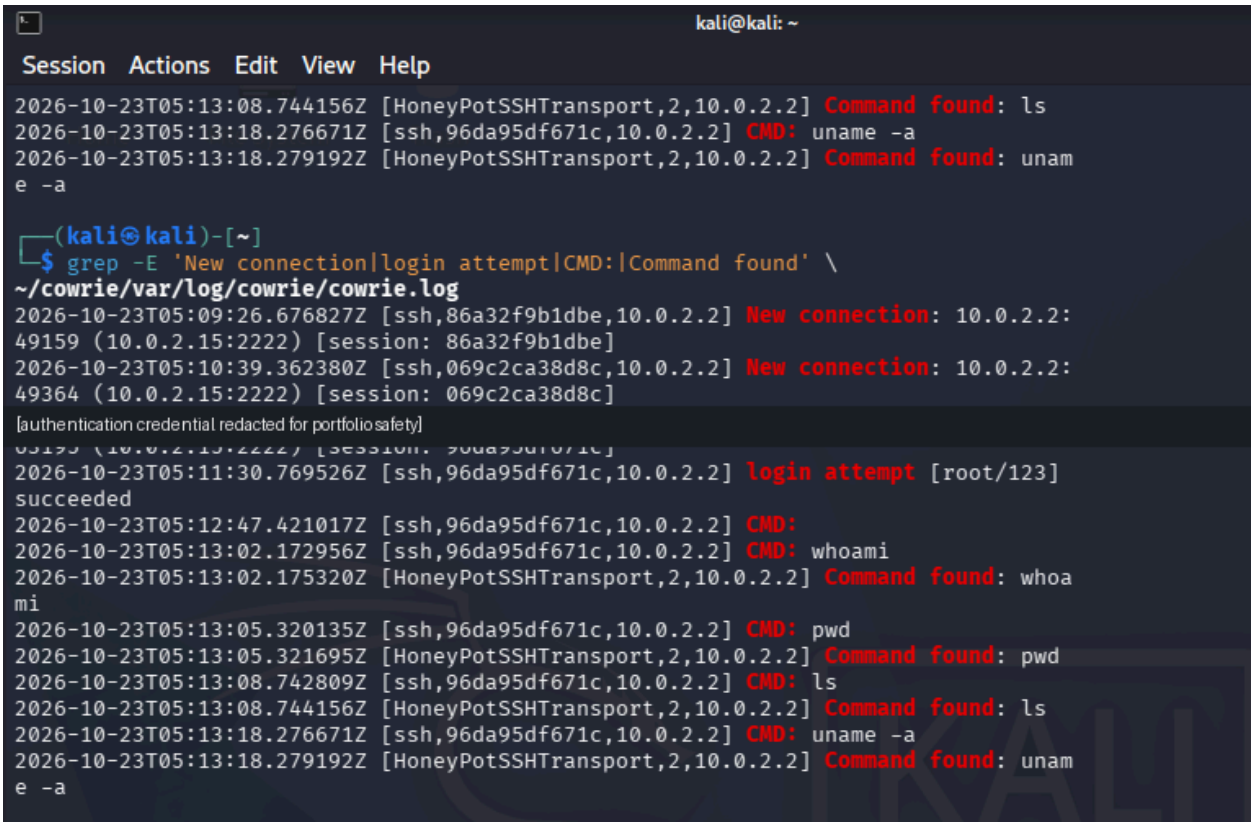
Structured JSON records show SSH connection/session details, including session identifiers, endpoint information, event identifiers, and timestamps. The screenshot demonstrates that machine-readable event logging was active.

Figure A3. Cowrie main service log



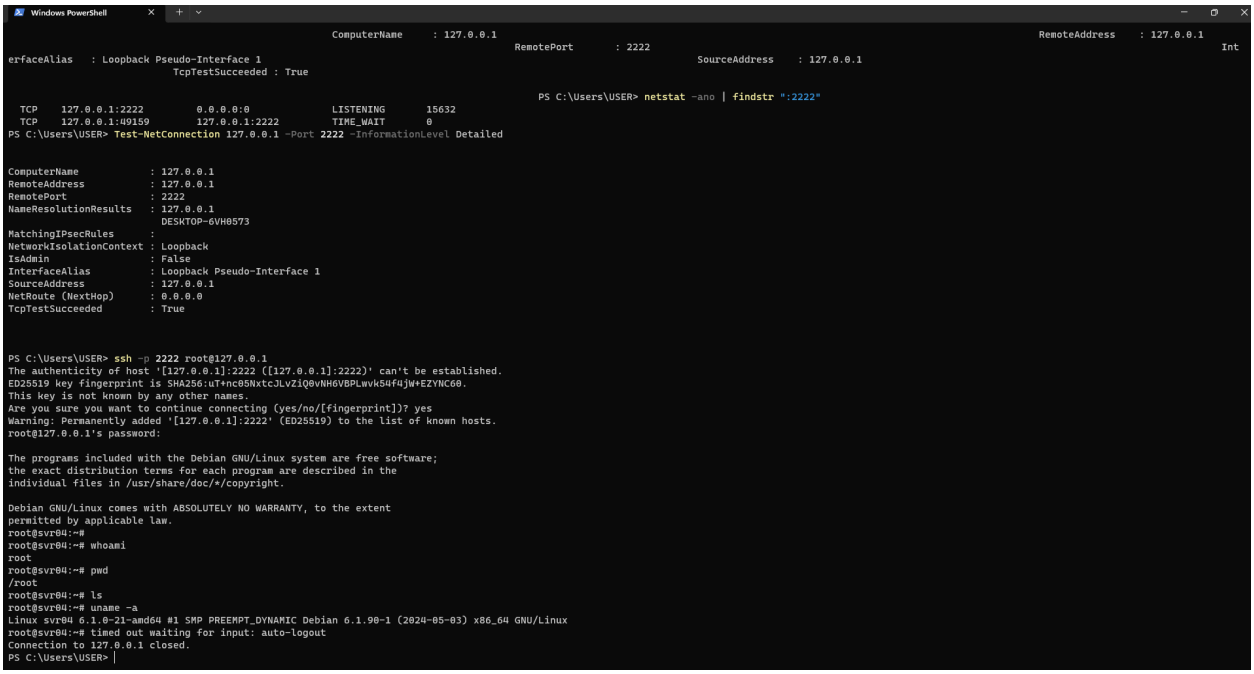
The main log shows Cowrie startup and subsequent SSH session activity. The authentication credential has been redacted in this report copy.

Figure A4. Filtered connection and command events



Filtered output highlights new connections, the simulated login event, and command records for whoami, pwd, ls, and uname -a. The credential line has been redacted.

Figure A5. Windows connectivity and SSH session



PowerShell shows a successful TCP test to 127.0.0.1:2222, a host-side listening socket, and the SSH session with the simulated prompt root@svr04:~#. The commands and output document the client-side validation.

Figure A6. Cowrie listening-port verification



Kali output from `ss -lntp` confirms that the `twistd` process was listening on TCP port 2222 at the time of capture.